实验 7-寄存器堆与 ALU 设计实验

一、 实验准备

1.1 装有 Vivado2020.1 的电脑一台：本实验不对 Vivado 环境有硬性要求，但涉及 Xilinx 库中 IP 的实验，在不同版本环境下无法兼容，且低版本 Vivado 无法运行高版本生成的项目，为方便实验检查，应当尽量使用实验要求版本。

1.2 熟悉 Vivado 的 IDE 环境，并能够使用其进行仿真、综合；（见第一次实验课件）

1.3 熟悉开发板；（见第一次实验课件）

二、 实验目的

2.1 了解算术逻辑单元 ALU 的原理；

2.2 熟悉并运用 Verilog 语言设计 ALU；

2.3 学习寄存器堆的数据传送与读写工作原理，掌握寄存器读写的设计方法；

2.4 熟悉并运用 Verilog 语言设计 ALU；

2.5 学习 Verilog 不同形式的编程方式，理解 assign 和 always 的区别；

三、 实验任务

2.1 ALU 设计实验

下图给出了一个具有 N 位输入和 N 位输出的算数逻辑单元的电路符号。算术逻辑单元接收说明执行哪个功能的控制信号 F，执行对应功能后输出 N 位结果。

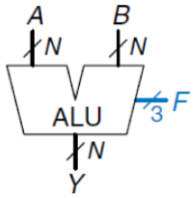


图 1

实验要求实现以下算术运算功能，其对应的指令码及功能如下：

表 1

$F_{2:0}$	功能	$F_{2:0}$	功能
000	A + B(Unsigned)	100	$\bar{A}$
001	A - B	***101***	***SLT***
010	A AND B	110	未使用
011	A OR B	111	未使用

本次实验将 ALU 输出结果通过板载串口进行显示验证，原理图如下图所示：

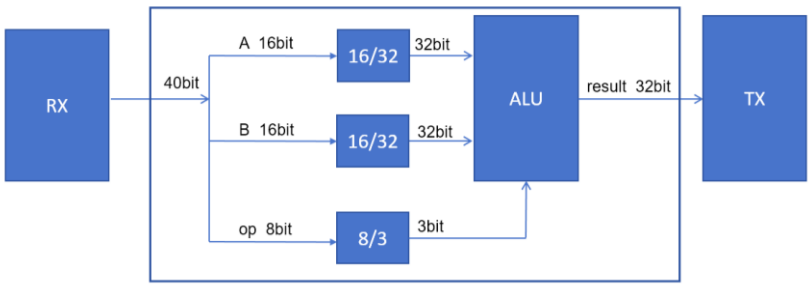


图 2

四、 实验要求

1. 根据 ALU 原理图(图 1)，使用 Verilog 语言定义 ALU 模块，其中输入输出端口参考实验原理，运算指令码长度为[2:0]。
2. 将串口收到的 40bit 数拆分为两个 16bit 数据与一个 8bit 数据，作为待计算数据 A、B 与操作符 op;
3. 将待计算数据 A、B 经过符号扩展至 32 位后，输入到运算器的端口 A、B;
4. 运算器支持"加、减、与、或"4 种运算，需要 3 位（8 个操作）。将操作符 op 截取低三位输入运算器的 op 端口作为运算器的控制信号(默认为高电平);
5.实现 SLT 功能，ALU 功能实现 Z，N，C，V 功能
6.将计算 32 位结果 s 输出到串口显示。
7.验证表 1 中所有功能。
8.给出 RTL 源程序（.v 文件）

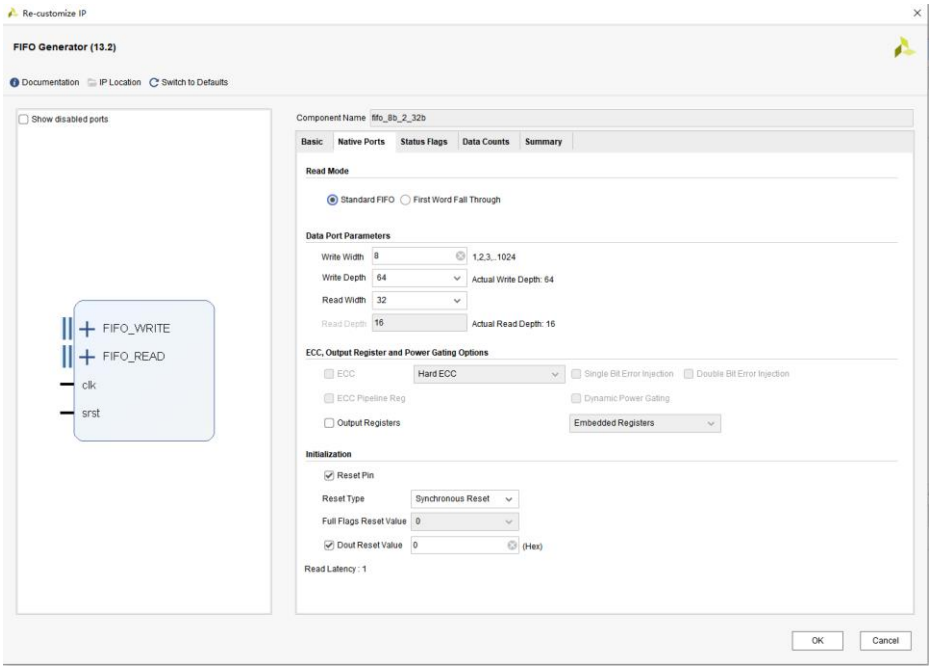
五、 实验步骤

1. 实现 alu.v 文件（本次实验重点），uart_rx.v，uart_tx.v (代码也已经提供)，在所提供的代码模板上加入 alu.v 文件。

2. 本次依然使用实验 6 中提供的串口模板，并提供了一个 alu_top.v 作为顶层模块，在其中包装好本次实验所需的位宽转换电路，将串口的 40bit 数据转换为 32bit 的 data_A、data_B 与 op，同学们只需要将这几个数据接到自己的 ALU 上进行测试，将输入接到 alu_result 信号上即可。
   你需要打开 alu_top.v，寻找 your ALU instantiation here 这行注释，并在下方例化你的 alu。一个 alu 的例化与连线样例如下，你的 ALU 至少需要包含这 4 个信号，并将端口连接到对应的数据线上，这样就可以使用串口测试你的 ALU 了。

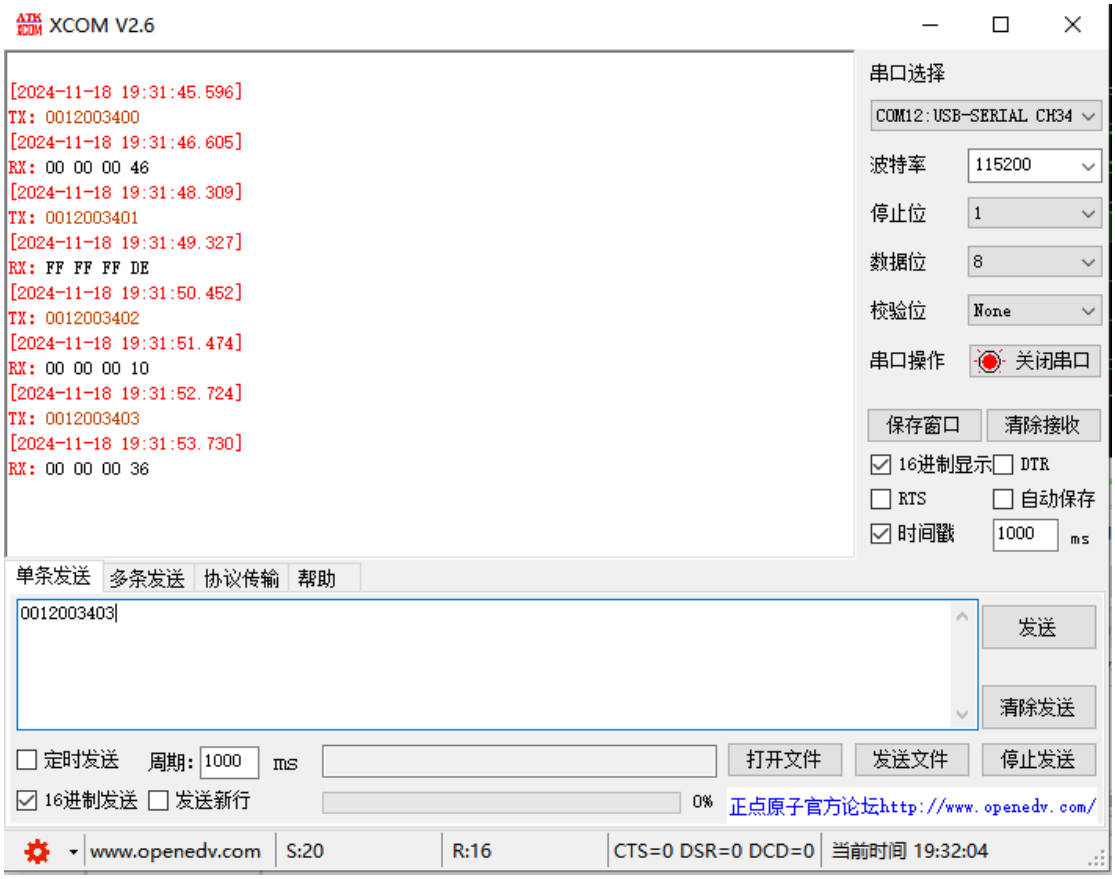
```
166     // your ALU instantiation here
167     alu u_alu(
168         .A_i      (data_A),
169         .B_i      (data_B),
170         .op_i     (op),
171         .result_o (alu_result)
172     );
173     // your ALU instantiation end
174
```

   其中所使用的位宽转换 fifo 配置如下：



本次实验中，tx_valid 会在数据传输完成两拍后拉高，请注意 alu 内的时序。如果有需求可以自行阅读代码修改打拍逻辑。

3. 生成比特流，使用串口调试助手输入测试数据。
输入数据格式为{A, B, op}，前 16 位为数据 A，中间 16 位为数据 B，最后 8 位为操作符 op，串口测试样例图如下所示：



其中：　op=00，　12 + 34= 46

　　op=01，　12 – 34 = FFFF FFDE

　　op=10，　12 & 34 = 10

　　op=11，　12 | 34 = 36

注意发送的数据是 16 进制，需要自己注意进制转换。

4. 熟悉寄存器堆

register file 给出了逻辑实现方式：

```
module regfile(
    input         clk,
    // READ PORT 1
    input  [ 4:0] raddr1,
    output [31:0] rdata1,
    // READ PORT 2
    input  [ 4:0] raddr2,
    output [31:0] rdata2,
    // WRITE PORT
    input         we,       //write enable, HIGH valid
    input  [ 4:0] waddr,
    input  [31:0] wdata
);
reg [31:0] rf[31:0];

//WRITE
always @(posedge clk) begin
    if (we) rf[waddr]<= wdata;
end

//READ OUT 1
assign rdata1 = (raddr1==5'b0) ? 32'b0 : rf[raddr1];

//READ OUT 2
assign rdata2 = (raddr2==5'b0) ? 32'b0 : rf[raddr2];

endmodule
```